

Gianluca Mariano Sobreiro
Guilherme Fornagiero De Carvalho
Paulo Vinícius Bessa De Brito
Pedro Augusto Bento Rocha

22.122.011-4
22.122.016-3
22.122.005-6
22.122.028-8

IMAGEM 1 - GIRAFA

Imagens Intermediárias

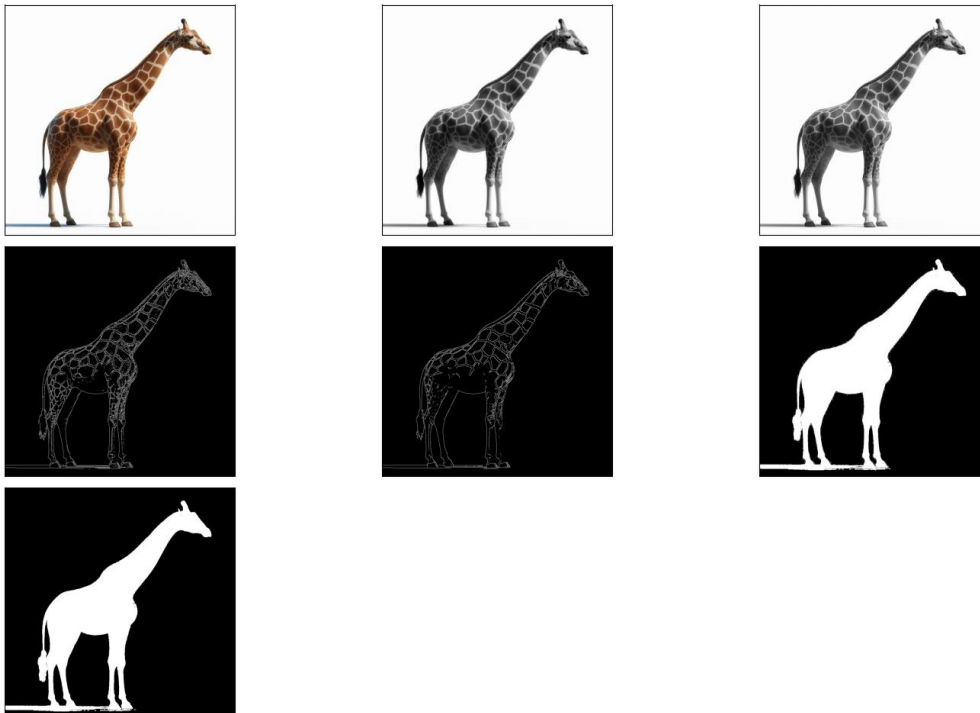


Imagem Final com Contornos Destacados

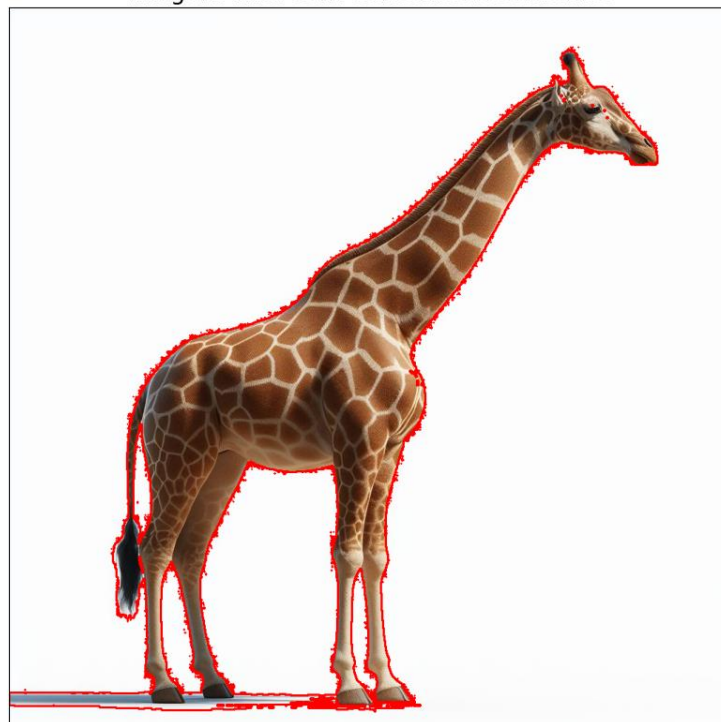


IMAGEM 2 – SATELITE

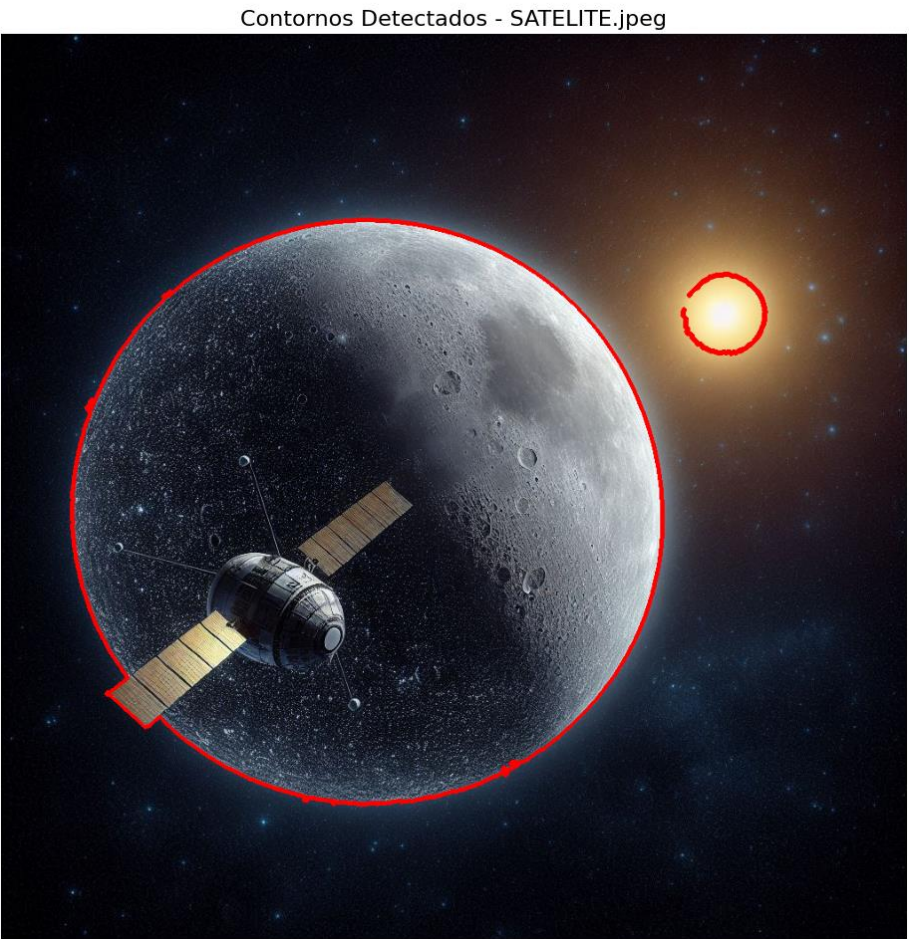
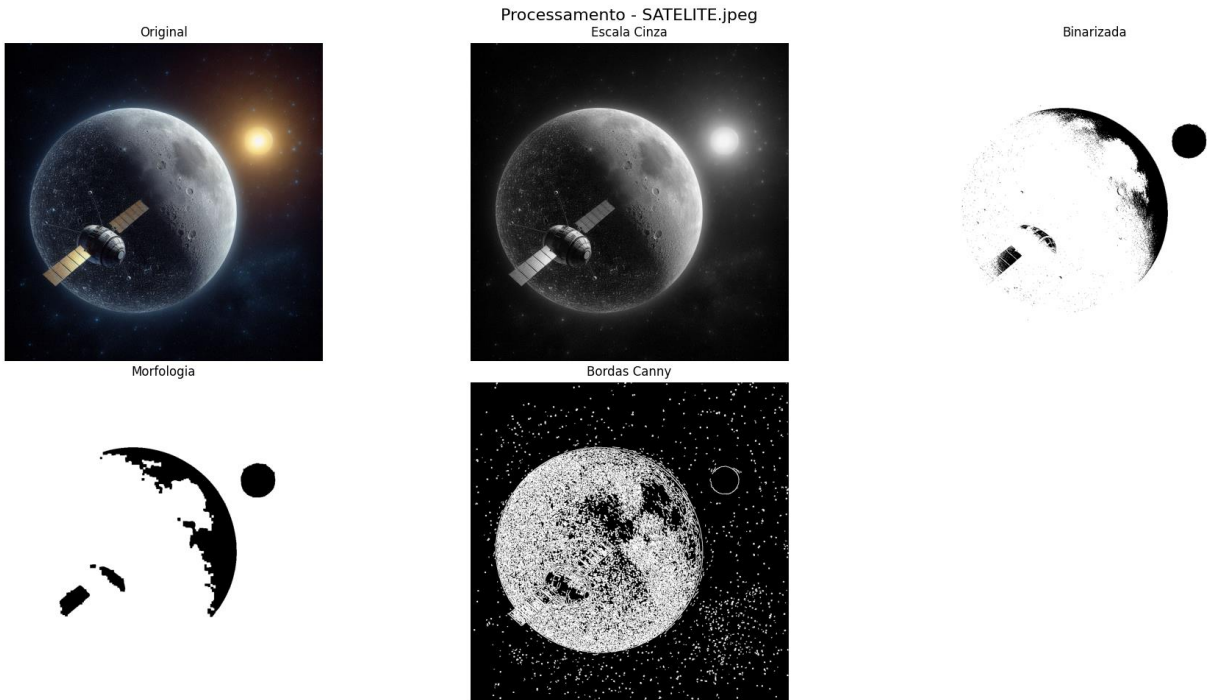
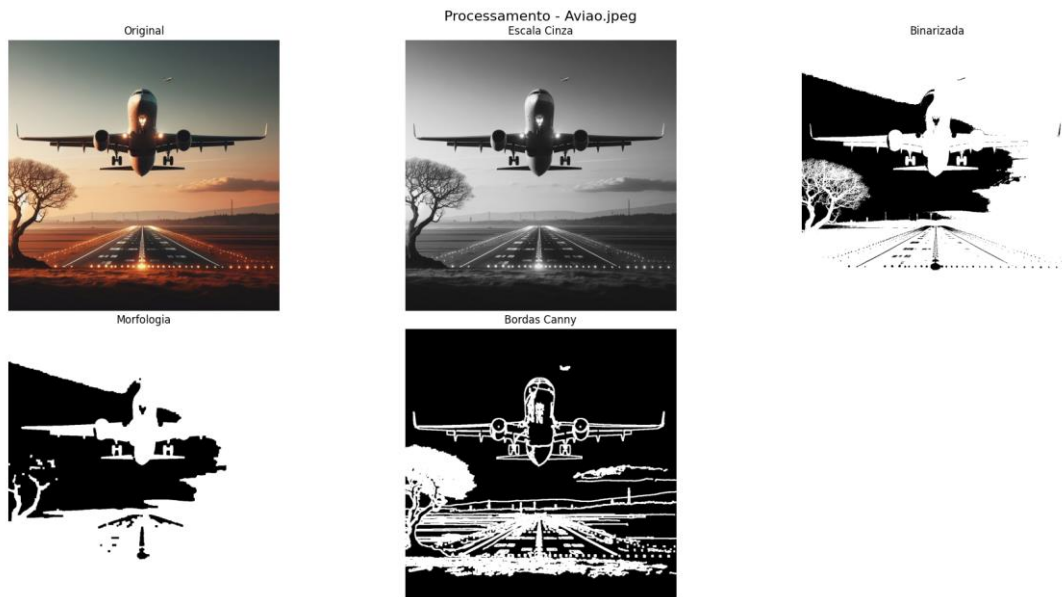
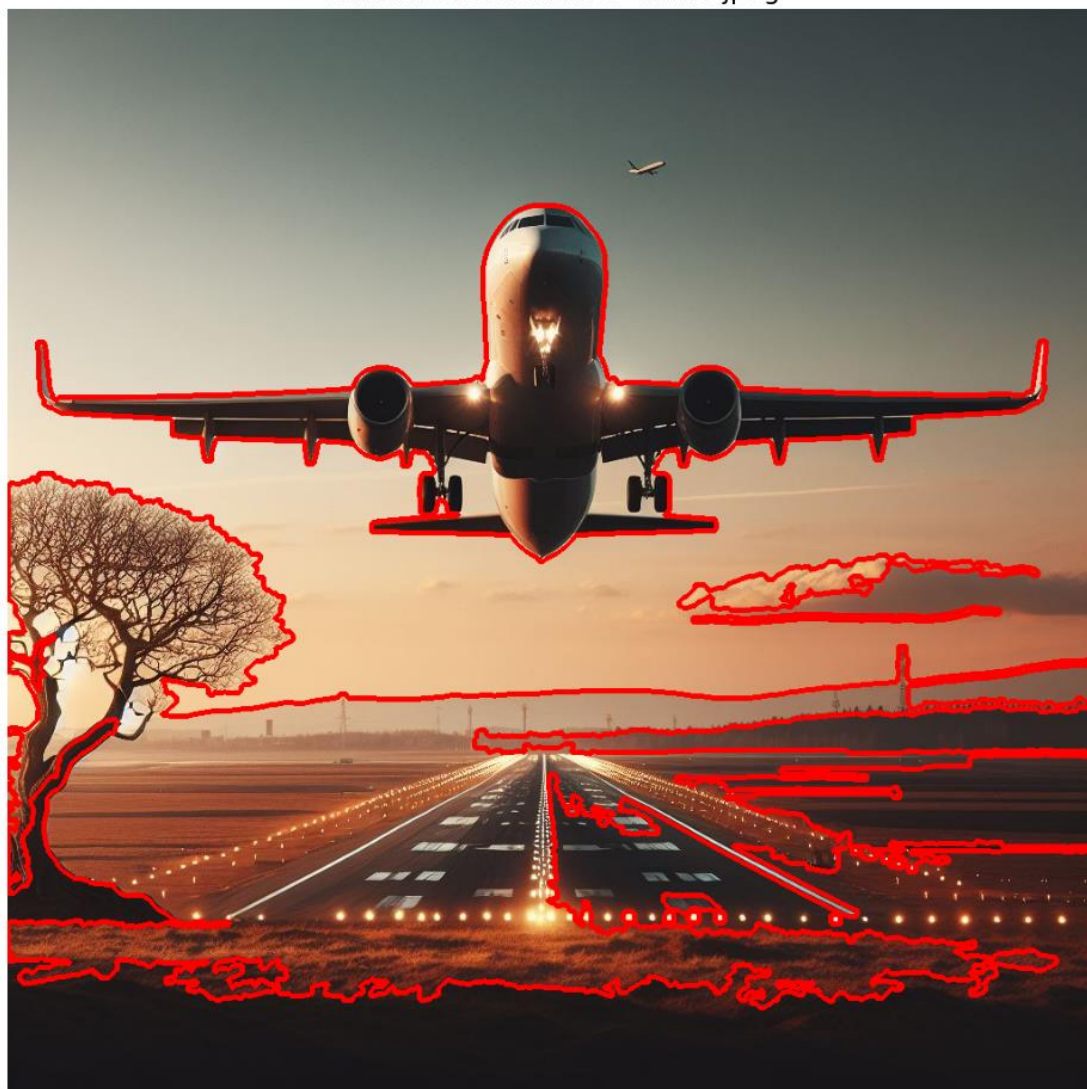


IMAGEM 3 – AVIÃO



Contornos Detectados - Aviao.jpeg



PASSO A PASSO

Neste relatório, apresento o processamento das imagens GIRAFA.jpeg, SATELITE.jpeg e AVIAO.jpeg, utilizando técnicas de visão computacional para detecção de bordas. O código foi desenvolvido em Python com OpenCV e Matplotlib.

1. Carregamento e Conversão de Cores

Objetivo: Carregar a imagem e ajustar o formato de cores para exibição correta.

Código:

python

Copy

```
img = cv2.imread(nome_arquivo)
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Explicação:

cv2.imread() carrega a imagem no formato BGR (padrão do OpenCV).

cv2.cvtColor() converte para RGB para visualização adequada no Matplotlib.

2. Conversão para Escala de Cinza

Objetivo: Simplificar o processamento reduzindo a dimensionalidade das cores.

Código:

python

Copy

```
img_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
```

Explicação:

A imagem é convertida para tons de cinza para facilitar operações de thresholding e detecção de bordas.

3. Binarização (Thresholding)

Objetivo: Destacar regiões de interesse.

Código:

python

Copy

```
_, thresh = cv2.threshold(img_gray, a/2*1.3, a, cv2.THRESH_BINARY_INV)
```

Explicação:

a = img_gray.max(): Obtém a intensidade máxima da imagem.

Limiar calculado como $a/2 * 1.3$ para inverter as cores (THRESH_BINARY_INV), destacando objetos claros em fundos escuros.

4. Operações Morfológicas

Objetivo: Remover ruídos e suavizar formas.

Código:

python

Copy

```
thresh_open = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
thresh_close = cv2.morphologyEx(thresh_open, cv2.MORPH_CLOSE, kernel)
```

Explicação:

Abertura (MORPH_OPEN): Remove ruídos pequenos (erosão seguida de dilatação).

Fechamento (MORPH_CLOSE): Preenche buracos em objetos (dilatação seguida de erosão).

kernel = np.ones((7,7), np.uint8): Define o tamanho da matriz para as operações.

5. Filtragem Gaussiana

Objetivo: Suavizar a imagem para reduzir ruídos.

Código:

python

Copy

```
img_blur = cv2.GaussianBlur(img_gray, (3,3), 0)
```

Explicação:

Um filtro Gaussiano com kernel 3x3 é aplicado para borrar a imagem, facilitando a detecção de bordas.

6. Detecção de Bordas (Canny)

Objetivo: Identificar bordas precisas.

Código:

python

Copy

```
edges = cv2.Canny(image=img_blur, threshold1=20, threshold2=70)
```

Explicação:

O algoritmo de Canny detecta bordas usando dois limiares:

threshold1 = 20: Limiar inferior para bordas fracas.

threshold2 = 70: Limiar superior para bordas fortes.

7. Dilatação das Bordas

Objetivo: Espessar as bordas detectadas.

Código:

python

Copy

```
edges = cv2.dilate(edges, kernel_edges, iterations=1)
```

Explicação:

Um kernel 5x5 é usado para dilatar as bordas, garantindo que os contornos sejam contínuos.

8. Detecção e Desenho de Contornos

Objetivo: Isolar objetos de interesse.

Código:

python

Copy

```
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

```
cv2.drawContours(img_contours, contours, -1, (255,0,0), 3)
```

Explicação:

cv2.findContours(): Encontra contornos externos na imagem de bordas.